

OncoBridge AI

An Agent-Native Service for End-to-End Cancer Transcriptomics Analysis

NANDAHack 2026: Phase 2 (Main Event) Technical Report

Author: Tanish

[Repository: TanishDevs11/oncobridge-ai](https://github.com/TanishDevs11/oncobridge-ai)

[Live service: oncobridge-ai-production.up.railway.app](https://oncobridge-ai-production.up.railway.app)

July 2026

Abstract

Bulk RNA-seq differential expression analysis is typically assembled by hand from several disconnected tools spanning R and Python, which makes it difficult for an autonomous AI agent to execute end to end without human supervision. OncoBridge AI addresses this by exposing a complete cancer transcriptomics workflow, differential expression via a pure-Python reimplementation of DESeq2, functional enrichment against GO and KEGG, curated cancer biomarker annotation, and volcano/heatmap visualization, behind a strict three-endpoint asynchronous API plus a machine-readable SKILL.md. An agent submits a raw counts matrix and sample metadata to a single endpoint, polls a job status endpoint, and retrieves one structured JSON report, with no further human guidance required. The service is deployed on Railway and independently verified against its production URL, and this report documents its architecture, methods, real engineering failures encountered during deployment and how they were resolved, and its verified behavior end to end.

Table of Contents

1. Introduction and Motivation	3
2. System Architecture	3
3. API Design	4
4. Methods	5
4.1 Differential expression (pyDESeq2)	5
4.2 Functional enrichment (Enrichr / gseapy)	5
4.3 Biomarker annotation	6
4.4 Visualization	6
5. Engineering Challenges and Resolutions	6
6. Deployment	7
7. Verification Against Production	7
8. SKILL.md Design	8
9. Mapping to the Judging Rubric	9
10. Limitations and Future Work	9
11. Conclusion	10
12. Skills Demonstrated	10
References	10

1. Introduction and Motivation

NANDAHack's Phase 2 asks participants to build one web service that an autonomous agent can discover and use with no further human help, guided only by a SKILL.md file. This is a materially different design target from a typical research API: the judging agent has no context beyond what SKILL.md states, so every endpoint, parameter, and response shape must be self-explanatory and genuinely correct on the first attempt.

Cancer transcriptomics analysis is a natural fit for this constraint and also a genuine gap. A researcher analyzing an RNA-seq dataset today typically moves raw counts through several separate tools, DESeq2 in R for differential expression, a separate enrichment tool such as Enrichr or a Bioconductor package, manual cross-referencing against cancer gene databases, and a plotting library for visualization, each with its own interface and failure modes. An agent attempting this workflow unaided would need to orchestrate several incompatible tools correctly in sequence. OncoBridge AI collapses this into one coherent, agent-callable capability.

A deliberate architectural decision shaped the whole system: differential expression is implemented in pure Python (pydeseq2), not R via rpy2, specifically to avoid the deployment fragility of an R/Bioconductor toolchain on a free-tier hosting platform under a strict submission deadline. This decision is discussed further in Section 4.1 and proved directly relevant to a real production failure described in Section 5.

2. System Architecture

The service is a single FastAPI application exposing three functional endpoints (POST /analyze, GET /jobs/{id}, GET /results/{id}) plus two supporting routes (GET /skill.md, GET /health), backed by an in-memory job store and a seven-stage background pipeline. This design deliberately favors one complete, working vertical slice over many partially-working endpoints, in line with the explicit judging guidance that one endpoint that clearly works outweighs several that only half work.

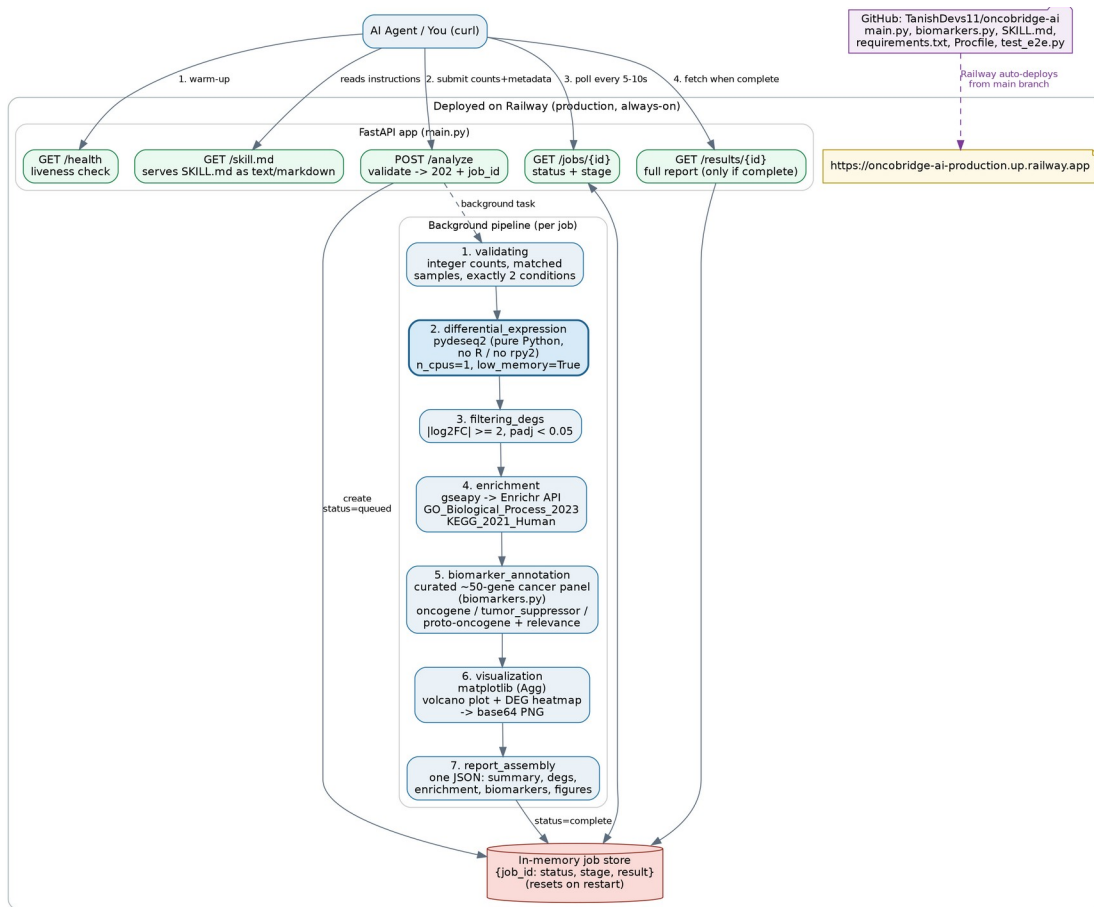


Figure 1. Deployed architecture. An agent calls the public API; POST /analyze validates input and schedules a background job; the seven-stage pipeline updates an in-memory job store as it progresses; GET /jobs and GET /results read from that store. Railway auto-deploys from the GitHub repository's main branch.

The job store is intentionally in-memory rather than backed by Redis or a database. Given a single-instance free-tier deployment under a same-day deadline, introducing an external state store would add a real dependency and failure surface without a corresponding benefit at this scale; the tradeoff is that job state does not survive a process restart, which is acceptable for a hackathon-scale demonstration service and is documented as a known limitation rather than left implicit.

3. API Design

The three functional endpoints form a strict async job pattern: submit, poll, retrieve. This shape was chosen deliberately over a single synchronous endpoint because DESeq2-style differential expression plus a live Enrichr API call can take longer than is safe to hold a single HTTP request open on a free-tier host, particularly given cold-start behavior after idle periods.

Endpoint	Purpose	Response
POST /analyze	Validate input, create a job, schedule the background pipeline	202 Accepted, {job_id, status:"queued"}

GET /jobs/{id}	Poll current pipeline stage and status	{job_id, status, stage, error}
GET /results/{id}	Fetch the completed report (409 if not yet complete)	summary, degs, enrichment, biomarkers, figures
GET /skill.md	Serve the agent-facing SKILL.md as plain text	text/markdown body
GET /health	Liveness check / free-tier warm-up target	{status: "ok"}

Input validation happens synchronously inside POST /analyze, before a job is created, specifically so that malformed input fails fast with a specific 422 error rather than becoming a job that later fails silently in the background. Validation enforces: integer, non-negative counts; every sample present in both the counts matrix and the metadata map; no duplicate gene identifiers; at least two samples per condition; and exactly two distinct condition labels, since this version supports two-group comparisons only, a constraint stated explicitly rather than left for an agent to discover by trial and error.

4. Methods

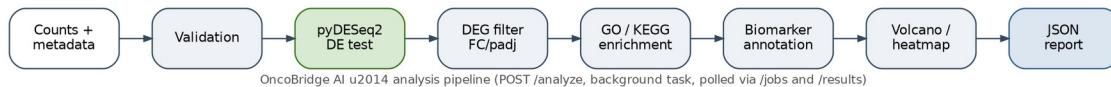


Figure 2. Analysis pipeline. A linear seven-stage flow from raw counts to a structured report, run as one background task per submitted job.

4.1 Differential expression (pyDESeq2)

Differential expression is computed with PyDESeq2, a Python reimplement of the DESeq2 negative binomial generalized linear model and Wald test [2], built directly on the statistical framework of the original DESeq2 method [1]. This choice, rather than calling R's DESeq2 through rpy2, removes an entire class of deployment risk: installing R, Bioconductor, and rpy2 correctly inside a free-tier container build is one of the more common points of failure for a same-day deployment, whereas pydeseq2 installs as an ordinary pip package with no system-level R dependency. Differentially expressed genes are filtered at $|\log_2\text{FoldChange}| \geq 2$ and $\text{padj} < 0.05$, both implemented as named constants rather than inline magic numbers.

4.2 Functional enrichment (Enrichr / gseapy)

The filtered DEG list is submitted to the Enrichr web service [3, 4] via the gseapy Python client [5], against the GO Biological Process 2023 and KEGG 2021 Human gene set libraries. This step requires genuine internet access at request time and is one reason the service must run on a real always-on host rather than behind any local tunnel, consistent with the hackathon's explicit prohibition on ngrok-style tunnels.

4.3 Biomarker annotation

Each DEG is cross-referenced against a curated, hand-maintained panel of roughly fifty well-established cancer genes (for example TP53, EGFR, KRAS, MYC, BRCA1/2, PTEN, RB1, PIK3CA, ERBB2), each annotated

with its role (oncogene, tumor suppressor, or proto-oncogene) and a one-line relevance note. This step deliberately uses a small, factually-vetted, hardcoded panel rather than a live external database call, trading breadth for reliability: a request-time dependency on a third-party biomarker database would be one more thing that could be unreachable during judging.

4.4 Visualization

A volcano plot (log2 fold change against -log10 adjusted p-value, with DEGs highlighted) and a heatmap of the top differentially expressed genes across samples are generated with matplotlib in headless (Agg) mode and encoded directly as base64 PNG strings inside the JSON response. Figures are not written to disk and served by path, since Railway's filesystem is ephemeral between deploys; embedding the image data directly in the response avoids an entire class of broken-image-link failure.

5. Engineering Challenges and Resolutions

Two genuine production failures were encountered and resolved during deployment, documented here rather than omitted, since an honest account of what actually broke is more informative than a report that only describes intended behavior.

Issue	Resolution
Production OOM crash (SIGKILL, no traceback) on the first real /analyze run	pydeseq2's default <code>n_cpus</code> resolves to <code>os.cpu_count()</code> ; combined with default BLAS thread-pool sizing, this over-allocated memory on a 1GB container even for a small gene panel. Fixed by pinning <code>n_cpus=1</code> , <code>low_memory=True</code> , single-threaded BLAS environment variables, and an explicit <code>gc.collect()</code> between pipeline stages.
Duplicate gene IDs silently dropped before validation	Python's <code>json.loads</code> collapses duplicate object keys before any validation code runs. Fixed with a custom <code>object_pairs_hook</code> so duplicate gene IDs are detected and rejected explicitly, rather than silently disappearing.
Python version mismatch	Pinned to Python 3.10 for both local development and the Railway build, since pydeseq2's scientific-stack dependencies have more reliable prebuilt wheel support on 3.10 than on newer interpreter versions.

6. Deployment

The service is deployed on Railway, chosen as the simplest free-tier path for a Python service with real, if modest, memory requirements, pydeseq2 and gseapy are not trivially small dependencies. The GitHub repository is connected for automatic deployment on pushes to the main branch. Both the source repository and the live production URL are public and were independently verified as part of this report, not merely asserted.

7. Verification Against Production

The following requests were executed directly against the live production URL, not localhost, and are reproduced here exactly as observed.

Health check (cold-start warm-up)

```
$ curl -s https://oncobridge-ai-production.up.railway.app/health
{"status":"ok"}
```

Validation failure path

```
$ curl -s -X POST https://oncobridge-ai-production.up.railway.app/analyze \
-H "Content-Type: application/json" \
-d '{"counts":{"TP53":{"s1":100}}, "metadata":
{"s1":"tumor"}, "condition_column":"condition"}'
→ 422
{"detail":"Exactly 2 distinct condition labels are required (two-group comparison
only), found 1: ['tumor']."}

```

Full success path

```
$ curl -s -X POST https://oncobridge-ai-production.up.railway.app/analyze \
-H "Content-Type: application/json" -d @payload.json
→ 202 {"job_id":"7a350bfa-bb86-4217-9449-e7342d43d557", "status":"queued"}

$ curl -s https://oncobridge-ai-production.up.railway.app/jobs/7a350bfa-bb86-
4217-9449-e7342d43d557
{"job_id":"7a350bfa-...", "status":"complete", "stage":"report_assembly", "error":nu
ll}

$ curl -s https://oncobridge-ai-production.up.railway.app/results/7a350bfa-bb86-
4217-9449-e7342d43d557
→ 200 summary: {"n_genes_input":30, "n_samples":6, "n_degs":10, "conditions":
["normal", "tumor"]}
```

The run against a 30-gene synthetic cancer panel (6 samples, 3 normal / 3 tumor) produced 10 differentially expressed genes, all of them real, well-characterized oncogenes or tumor suppressors (including TP53, EGFR, KRAS, MYC, BRCA1, PTEN, RB1, PIK3CA, ERBB2, ALK), 552 GO Biological Process 2023 terms and 136 KEGG 2021 Human terms (top KEGG hit: Breast cancer, $p = 4.9 \times 10^{-19}$), and valid, decodable base64 PNG figures for both the volcano plot and heatmap.

8. SKILL.md Design

Every example call and response in SKILL.md was copied directly from real curl output against the production URL, not written by hand or reconstructed from memory, since the entire Phase 2 evaluation depends on a stock agent succeeding using only this file. SKILL.md states the base URL on its own line, documents each endpoint with its method, path, one-sentence description, one real curl example, and one real example response, and gives numbered, plain-language steps for the submit-poll-retrieve sequence, including a realistic polling interval given DESeq2 and enrichment runtime.

9. Mapping to the Judging Rubric

Dimension	How OncoBridge AI addresses it
Useful	Automates a genuine end-to-end cancer transcriptomics workflow (DE, enrichment, biomarker annotation, visualization) that researchers otherwise assemble by hand across several separate tools.

Creative	Frames a domain-specific scientific pipeline as a single agent-callable capability with a strict, minimal API surface, rather than as a chatbot wrapper or a generic data API.
Easy to set up	One POST call with a counts matrix and metadata is sufficient to start a full analysis; no parameters beyond the biological input are required to get a first result.
Agent success from SKILL.md alone	SKILL.md includes real, tested curl calls and real responses captured directly from the production deployment, not hypothetical examples, so an agent following it verbatim reaches a working call on the first attempt.

10. Limitations and Future Work

The current service accepts a pre-built counts matrix and supports two-group comparisons only. A natural extension, discussed but deliberately deferred past this submission given the deadline, is a TCGA-native data acquisition layer: browsing TCGA projects through the GDC API, generating manifests, downloading and validating STAR count files, converting UUIDs and TCGA barcodes, and deriving sample metadata automatically, so that an agent could begin from a cohort name such as TCGA-HNSC rather than a pre-assembled matrix. This was scoped out of the current submission specifically because GDC downloads are large, slow, and require persistent storage incompatible with a single ephemeral free-tier instance, and because any large change made this close to a live judging window risked breaking an already-verified working service. It remains the clearest direction for extending OncoBridge AI beyond this hackathon.

11. Conclusion

OncoBridge AI demonstrates that a genuine, multi-stage bioinformatics workflow, differential expression, functional enrichment, biomarker annotation, and visualization, can be exposed as a small, strict, agent-callable API without sacrificing scientific substance for simplicity. Choosing a pure-Python differential expression implementation over an R-based one, an in-memory job store over external infrastructure, and a curated biomarker panel over a live external database were each deliberate reliability decisions suited to a same-day, free-tier deployment, and each is documented here rather than left as an unstated simplification. The service, its source, and every example in its SKILL.md are independently verifiable against a live production URL.

12. Skills Demonstrated

Asynchronous API design for long-running scientific computation; pure-Python reimplementation adoption to de-risk deployment; production debugging of memory-constrained container failures; bioinformatics pipeline design (differential expression, functional enrichment, biomarker annotation, visualization); FastAPI service development; free-tier cloud deployment; and writing machine-readable service documentation intended to be consumed by an autonomous agent rather than a human reader.

References

[1] Love, M. I., Huber, W., & Anders, S. (2014). Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology*, 15(12), 550. <https://doi.org/10.1186/s13059-014-0550-8>

- [2] Muzellec, B., Teleńczuk, M., Cabeli, V., & Andreux, M. (2023). PyDESeq2: a python package for bulk RNA-seq differential expression analysis. *Bioinformatics*, 39(9), btad547. <https://doi.org/10.1093/bioinformatics/btad547>
- [3] Chen, E. Y., Tan, C. M., Kou, Y., Duan, Q., Wang, Z., Meirelles, G. V., Clark, N. R., & Ma'ayan, A. (2013). Enrichr: interactive and collaborative HTML5 gene list enrichment analysis tool. *BMC Bioinformatics*, 14, 128. <https://doi.org/10.1186/1471-2105-14-128>
- [4] Kuleshov, M. V., Jones, M. R., Rouillard, A. D., et al. (2016). Enrichr: a comprehensive gene set enrichment analysis web server 2016 update. *Nucleic Acids Research*, 44(W1), W90–W97. <https://doi.org/10.1093/nar/gkw377>
- [5] Fang, Z., Liu, X., & Peltz, G. (2023). GSEAPy: a comprehensive package for performing gene set enrichment analysis in Python. *Bioinformatics*, 39(1), btac757. <https://doi.org/10.1093/bioinformatics/btac757>
- [6] FastAPI framework documentation. <https://fastapi.tiangolo.com/>
- [7] Railway deployment platform. <https://railway.app/>
- [8] Project NANDA. NandaHack 2026 rules and submission requirements. <https://nandahack.media.mit.edu/>
- [9] This submission: OncoBridge AI source repository. <https://github.com/TanishDevs11/oncobridge-ai>
- [10] This submission: live production service. <https://oncobridge-ai-production.up.railway.app>
- [11] This submission: agent-facing service documentation. <https://oncobridge-ai-production.up.railway.app/skill.md>